Prakash Thapa
D20A- 32

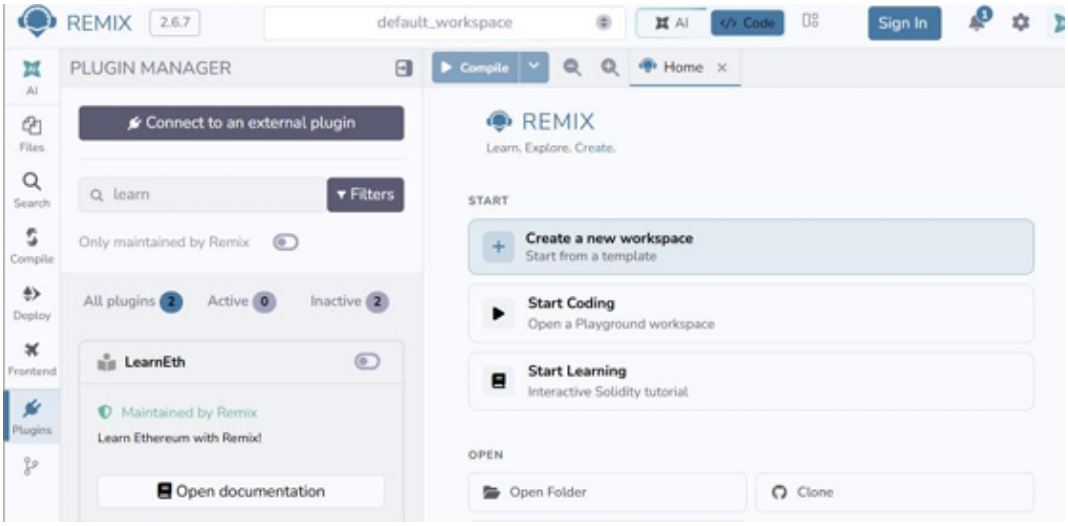
Experiment No.: 9
Create an ERC-20 Token in Remix IDE

1. Aim
To create and deploy an ERC-20 fungible token using Solidity in Remix IDE and understand the ERC-20 token standard, its functions, events, attributes, and extensions such as Mintable, Burnable, and Pausable.

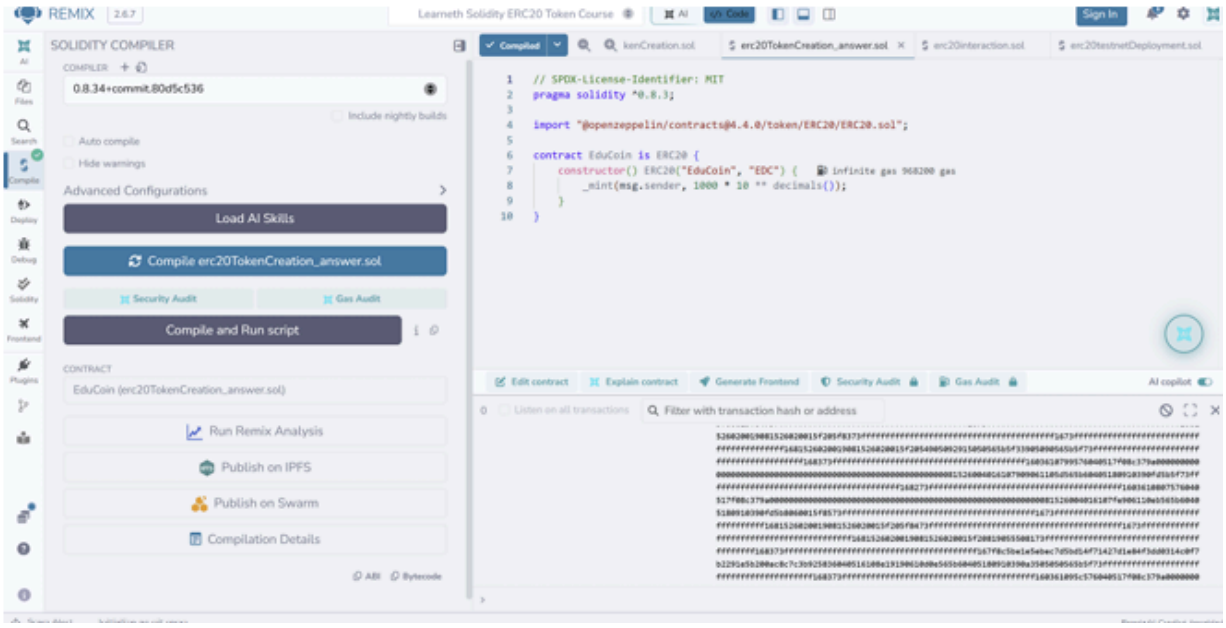
2. Theory
An ERC-20 token is a fungible digital token created on the Ethereum blockchain using smart contracts. ERC stands for Ethereum Request for Comments, and 20 is the identification number of the token standard.
ERC-20 defines a common set of rules and functions that allow different tokens to interact with wallets, decentralized applications (DApps), and cryptocurrency exchanges.
Each ERC-20 token has identical units, meaning one token is interchangeable with another token of the same type. These tokens can represent digital currencies, utility tokens, governance rights, or other blockchain-based assets.
Remix IDE is an online Integrated Development Environment used to write, compile, deploy, and interact with Solidity smart contracts. It allows developers to deploy contracts on local environments as well as public testnets using MetaMask.
MetaMask is a cryptocurrency wallet that allows users to interact with Ethereum-compatible blockchain networks, manage tokens, and approve blockchain transactions.

3. Tasks Performed
The following tasks were performed to create and deploy an ERC-20 token:

Step 1: Open Remix IDE
Opened Remix IDE using the following link:
https://remix.ethereum.org/
Explored the LearnETH tutorials and completed the six chapters of the Solidity ERC-20 Token Course to understand token creation and smart contract functionality.



Step 2: Create a Solidity Smart Contract
Created a new Solidity file named MyToken.sol in the Remix IDE workspace. Selected Solidity version 0.8.18 and implemented an ERC-20 token smart contract using the OpenZeppelin ERC-20 library.

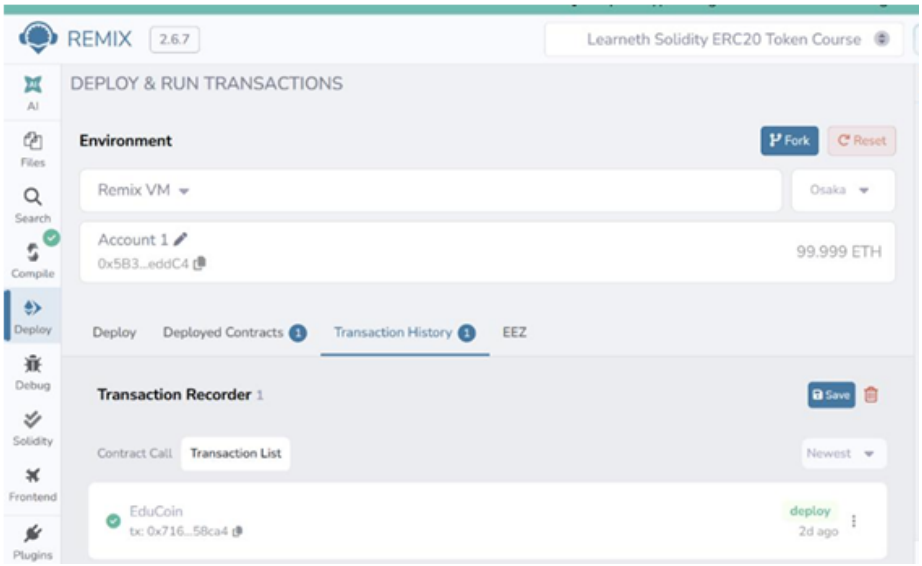


Step 3: Create MetaMask Wallet
Installed and configured the MetaMask browser extension. Created a new wallet and securely stored the recovery phrase.

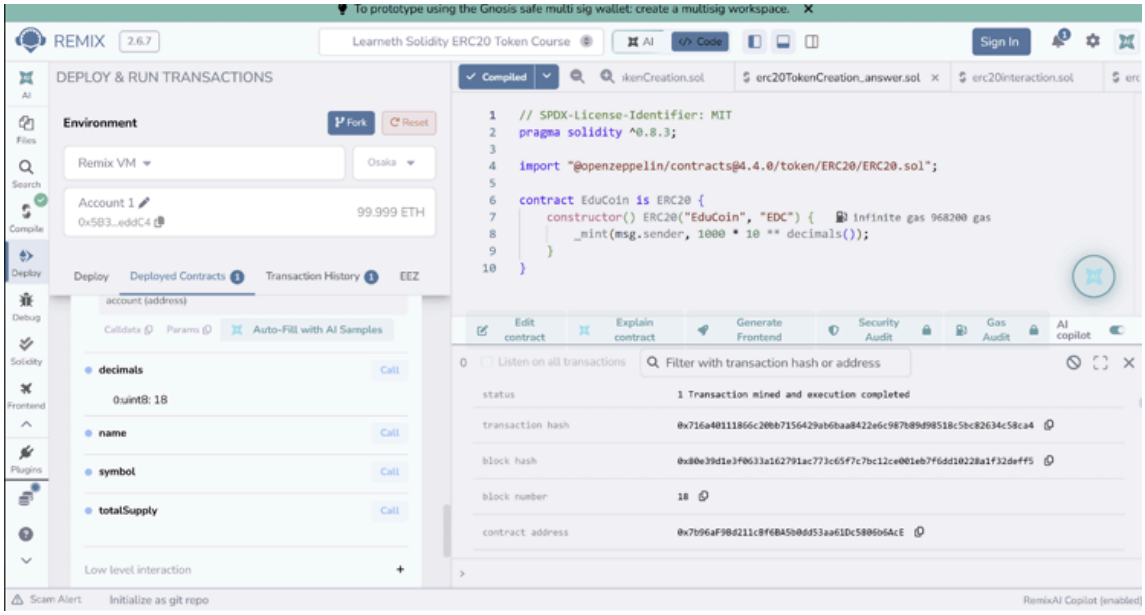
Step 4: Configure Testnet
Connected MetaMask to the Sepolia Ethereum Testnet and selected the same network in Remix IDE using the Injected Provider option.

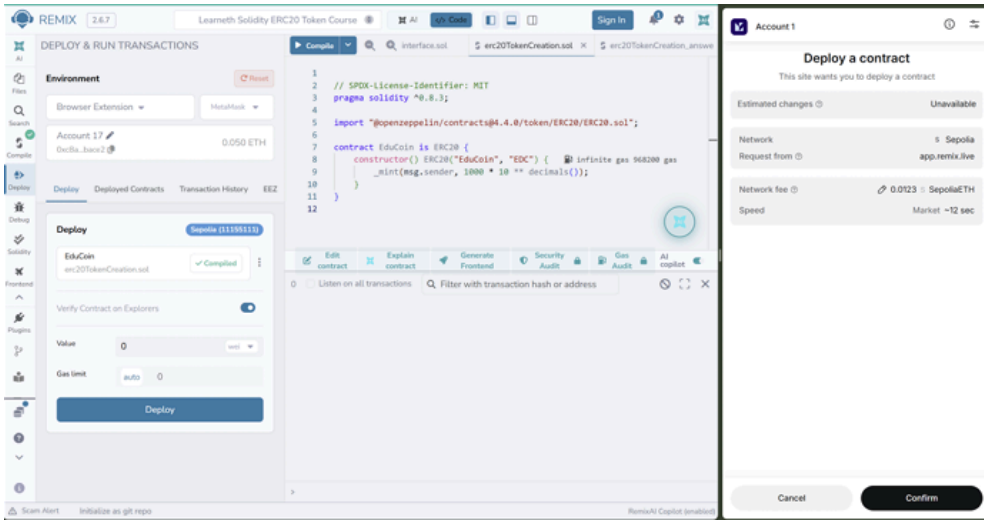
Step 5: Fund the Wallet
Obtained Sepolia test ETH from an available testnet faucet to pay for smart contract deployment and transaction gas fees.

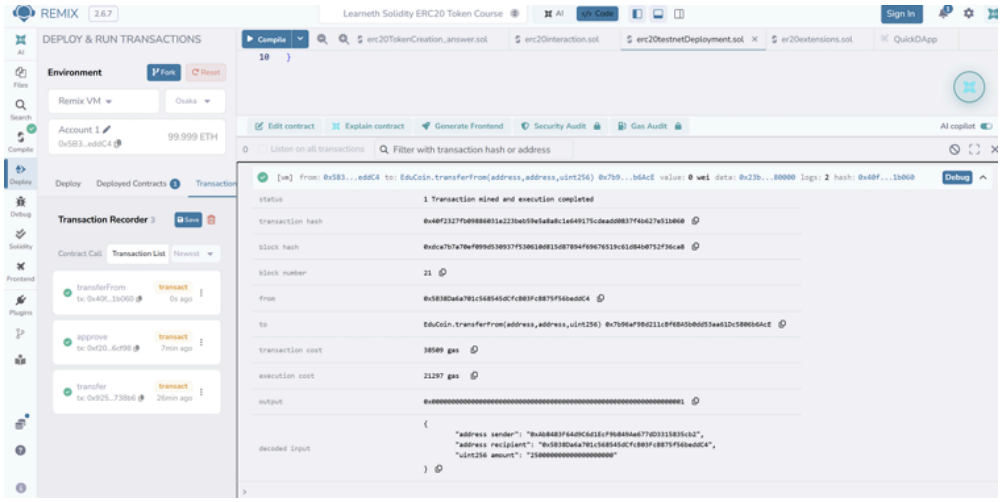
Step 6: Compile the Smart Contract
Selected Solidity compiler version 0.8.18 in Remix IDE and compiled the smart contract. Verified that the compilation was successful without errors.



Step 7: Deploy the ERC-20 Token
Selected Injected Provider – MetaMask in the Deploy & Run Transactions section. Selected the MyToken contract and deployed it to the Sepolia Testnet by confirming the transaction through MetaMask.



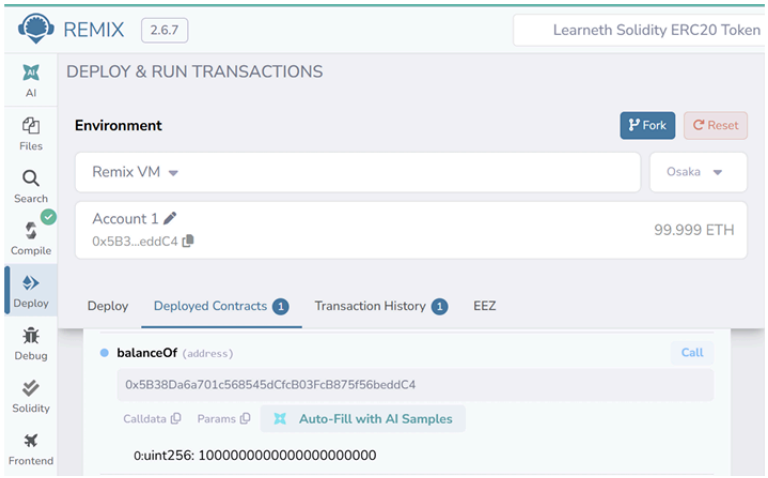


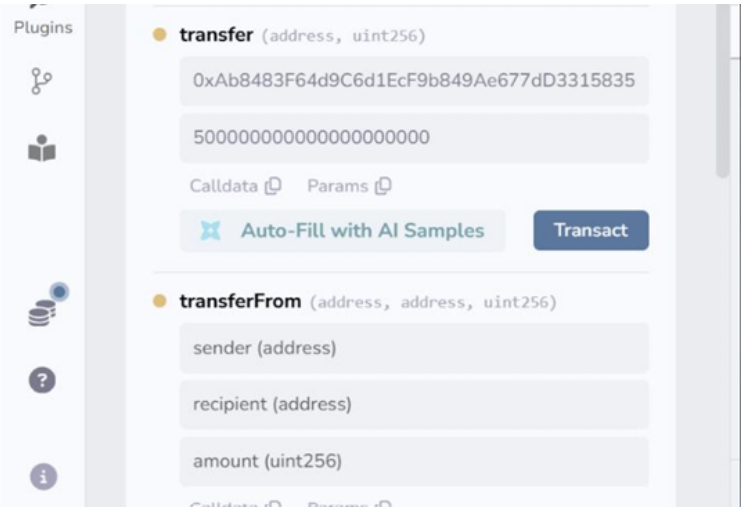


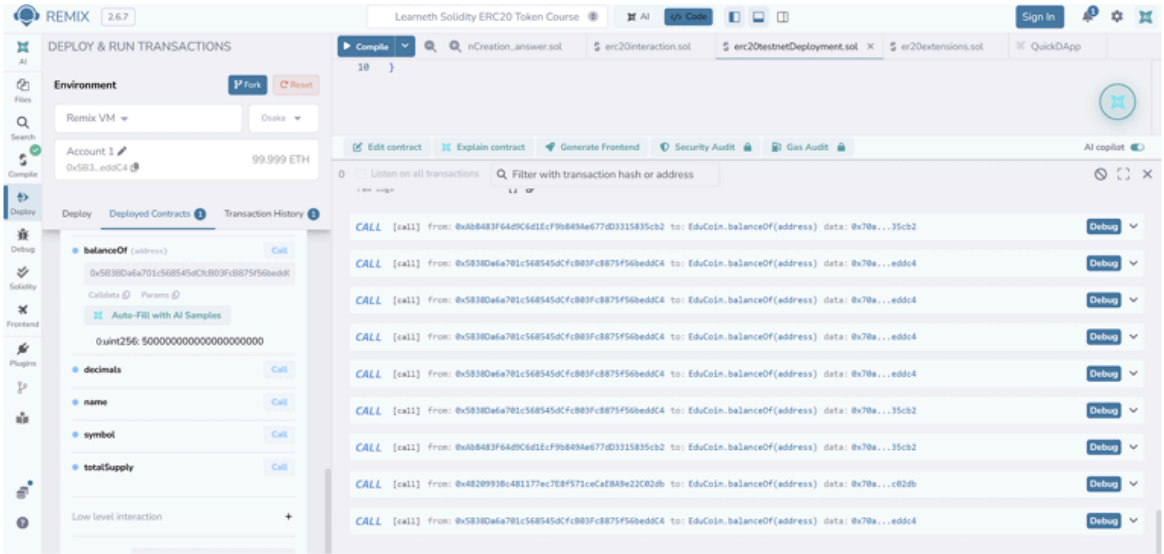
Step 8: Interact with the Deployed Token

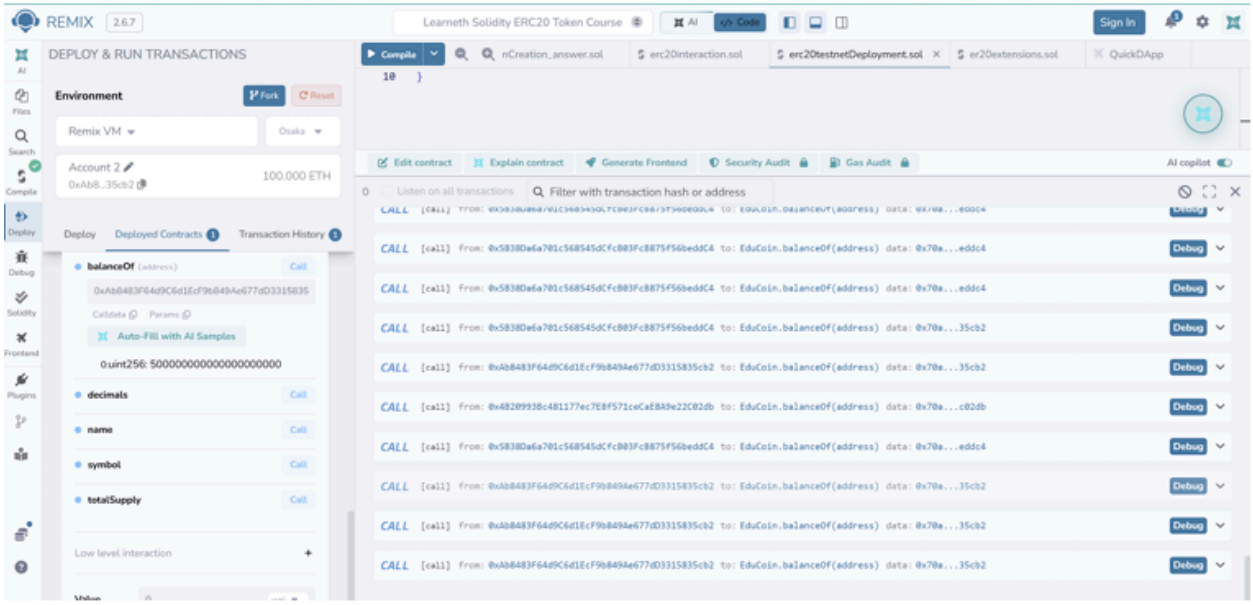
After successful deployment, interacted with the smart contract using its available functions, such as:

- name() – Retrieves the token name.
- symbol() – Retrieves the token symbol.
- totalSupply() – Returns the total token supply.
- balanceOf() – Checks the token balance of an address.
- transfer() – Transfers tokens to another address.
- approve() – Approves another address to spend tokens.
- allowance() – Checks the remaining approved spending amount.


REMIX
2.6.7
Learneth Solidity ERC20 Token
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 1
EEZ
balanceOf (address)
Call
0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 100000000000000000000


Plugins
transfer (address, uint256)
0xAb8483F64d9C6d1EcF9b849Ae677dD3315835
50000000000000000000
Calldata
Params
Auto-Fill with AI Samples
Transact
transferFrom (address, address, uint256)
sender (address)
recipient (address)
amount (uint256)


REMIX
2.6.7
Learneth Solidity ERC20 Token Course
Sign In
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 1
balanceOf (address)
Call
Auto-Fill with AI Samples
0:uint256: 50000000000000000000
decimals
name
symbol
totalSupply
Low level interaction
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Filter with transaction hash or address
CALL
Debug



4. Program

The following Solidity program creates a basic ERC-20 token using the OpenZeppelin library.

File Name: MyToken.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MyToken is ERC20 {

    constructor() ERC20("MyToken", "MTK") {
        _mint(msg.sender, 1000000 * 10 ** decimals());
    }
}
```

Program Explanation:

1. SPDX-License-Identifier: Specifies the license under which the smart contract is distributed.
2. pragma solidity ^0.8.18: Specifies the compatible Solidity compiler version.
3. import: Imports the ERC-20 implementation from the OpenZeppelin library.
4. contract MyToken is ERC20: Creates a smart contract that inherits ERC-20 functionality.
5. constructor(): Executes automatically when the contract is deployed.
6. ERC20("MyToken", "MTK"): Sets the token name as MyToken and symbol as MTK.
7. _mint(): Creates an initial supply of 1,000,000 tokens and assigns them to the deploying wallet.

8. decimals(): Returns the default decimal precision of 18.

Note: The program uses OpenZeppelin's ERC-20 implementation, so standard functions such as transfer, approve, and transferFrom are already implemented.

5. ERC-20 Token Functions, Events and Attributes

A. Functions

Function	Description
totalSupply()	Returns the total number of tokens in circulation.
balanceOf(address)	Returns the token balance of a given address.
transfer(address, amount)	Transfers tokens to another address.
approve(address, amount)	Allows another address to spend tokens on behalf of the owner.
allowance(owner, spender)	Returns the remaining approved spending amount.
transferFrom()	Transfers tokens using an existing approval.

B. Events

Event	Description
Transfer	Emitted whenever tokens are transferred, including minting and burning.
Approval	Emitted whenever an allowance is set or updated.

C. Attributes

Attribute	Description
name	Specifies the full name of the token.
symbol	Specifies the token's abbreviation.
decimals	Defines the number of decimal places used by the token.
totalSupply	Represents the total number of tokens currently existing.
balanceOf	Represents the balance of a particular wallet address.

6. Difference Between ERC-20 and ERC-721

ERC-20 and ERC-721 are two important Ethereum token standards used for different purposes.

Parameter	ERC-20	ERC-721

Token Type	Fungible	Non-fungible
Interchangeability	Tokens are interchangeable	Each token is unique
Identification	Tokens share the same contract identity	Each token has a unique Token ID
Divisibility	Supports decimal units	Generally indivisible
Main Purpose	Digital currencies and utility tokens	NFTs and digital collectibles
Examples	USDT, LINK	CryptoKitties, digital art NFTs
Transfer	Transfers a specified amount	Transfers a specific token ID

7. ERC-20 Token Extensions

ERC-20 tokens can be extended with additional functionalities depending on the application's requirements.

A. Mintable Extension

Mintable allows authorized users, usually the contract owner or designated minters, to create new tokens and increase the total supply.

- Creates additional tokens.
- Increases the total token supply.
- Useful for reward systems and controlled token issuance.

B. Burnable Extension

Burnable allows tokens to be permanently destroyed, reducing the total token supply.

- Removes tokens from circulation.
- Decreases the total supply.
- Useful for token supply management and deflationary mechanisms.

C. Pausable Extension

Pausable allows authorized users to temporarily stop certain token operations.

- Restricts transfers while the contract is paused.
- Helps prevent unauthorized activity during emergencies.
- Allows normal operations to resume after unpausing.

Extension	Main Function	Effect
Mintable	Creates new tokens	Increases supply
Burnable	Destroys existing tokens	Decreases supply

Pausable	Temporarily stops operations	Restricts token transfers

8. Output

The ERC-20 smart contract was successfully created, compiled, and deployed using Remix IDE on the Sepolia Ethereum Testnet through MetaMask.
The deployed smart contract provided the standard ERC-20 functionalities, including checking token balances, transferring tokens, approving spending allowances, and retrieving the total token supply.
Screenshots to attach:

1. Remix IDE workspace containing the Solidity smart contract.
2. Successful compilation of the smart contract.
3. MetaMask wallet connected to the Sepolia Testnet.
4. MetaMask wallet showing the available test ETH.
5. Deployment transaction confirmation in MetaMask.
6. Successfully deployed contract displayed in Remix IDE.
7. Token functions showing the name, symbol, total supply, and balance.
8. Successful token transfer transaction (if performed).

9. Conclusion

Thus, the experiment was successfully performed to create and deploy an ERC-20 token using Solidity in Remix IDE. The experiment provided practical knowledge of smart contract development, compilation, deployment, and interaction with the Ethereum Sepolia Testnet using MetaMask.
We also understood the ERC-20 token standard, its important functions, events, and attributes, along with the differences between ERC-20 and ERC-721 standards. Additionally, we studied the Mintable, Burnable, and Pausable extensions and their applications in token management.